

Lecture-12

Virtual Functions and Polymorphism

Virtual Functions and Polymorphism

- Polymorphism in C++ is manifested in both compile-time and run-time scenarios.
- Compile-time polymorphism is achieved through *function overloading* and *operator overloading*, while run-time polymorphism is enabled through *inheritance* and *virtual functions*.
- **Virtual functions**, essential for run-time polymorphism, are member functions declared within a base class and redefined in derived classes to suit their specific requirements. This redefinition facilitates the “**one interface, multiple methods**” approach inherent in polymorphism.
- Normally, virtual functions behave similar to other class member functions. However, their significance in supporting run-time polymorphism becomes apparent when accessed via pointers.
- When a base class pointer points to a derived object containing a virtual function, C++ dynamically determines which version of the function to invoke based on the type of object referenced by the pointer.
- Upon initial inspection, the redefinition of a virtual function in a derived class might seem reminiscent of function overloading. However, this assumption is misleading, as there are notable distinctions, leading to the avoidance of the term ‘*overloading*’ in this context.

Virtual Functions and Polymorphism – *cont.*

- One significant difference lies in the necessity for the redefined function to precisely match the prototype specified in the base class. If there's any deviation from this prototype during the attempt to redefine a virtual function, the C++ compiler will simply treat the function as overloaded, thereby losing its virtual nature.
- Given these constraints and differences between function overloading and virtual function redefinition, the term '**overriding**' is utilized to describe the process of a derived class redefining a virtual function.
- This term emphasizes the replacement of the base class's implementation with a specialized version in the derived class, maintaining the polymorphic behaviour essential for run-time dispatch.
- Polymorphism provides several benefits in object-oriented programming:
 - ❑ **Flexibility**: allows you to write more flexible and adaptable code
 - ❑ **Code reusability**: promotes code reuse by enabling you to write generic code that can operate on a wide range of object types
 - ❑ **Extensibility**: facilitates adding new classes and behaviours to your codebase without needing to modify existing code
 - ❑ **Encapsulation**: helps enforce encapsulation by allowing you to define a public interface for interacting with objects, while hiding the implementation details
 - ❑ **Improved readability and maintainability**: can make your code more readable and understandable by promoting a more abstract and high-level view of the system

Lecture-12

COMP218 by Dr. Ferhun Yorgancıoğlu

3

Example-1: Virtual Functions

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw() { // Virtual function
        cout << "Drawing a general shape." << endl;
    }
};
class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle shape." << endl;
    }
};
class Square : public Shape {
public:
    void draw() {
        cout << "Drawing a square shape." << endl;
    }
    // Below is an overloaded function
    void draw(int) {
        cout << "Drawing square x2." << endl;
    }
};
```

virtual functions

The **virtual** attribute of a function is inherited by derived classes, allowing them to override the function and enabling polymorphic behaviour.

```
int main() {
    Shape shape;
    // Just a member function call
    shape.draw();

    Circle circle;
    // Just a member function call
    circle.draw();

    Square square;
    // Just a member function call
    square.draw();
    // The overloaded function is called
    square.draw(1);

    return 0;
}
```

Microsoft Visual Studio Debug

```
Drawing a general shape.
Drawing a circle shape.
Drawing a square shape.
Drawing square x2.
```

If the same function prototype is used, it leads to a virtual function.
However, if the prototype changes, it is considered overloaded.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

4

Example-2: Virtual Functions and Polymorphism

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw() { // Virtual function
        cout << "Drawing a general shape." << endl;
    }
};
class Circle : public Shape {
public:
    // virtual keyword in below line is unnecessary
    void draw() {
        //virtual void draw() {
        cout << "Drawing a circle shape." << endl;
    }
};
class Square : public Shape {
public:
    // The same prototype should repeat, however.
    // Otherwise, it is considered overloaded.
    void draw() {
        cout << "Drawing a square shape." << endl;
    }
};
```

```
int main() {
    Shape *p; // Base class pointer

    Circle circle;
    // Base class pointer at derived class object
    p = &circle;
    p->draw();

    Square square;
    // Base class pointer at derived class object
    p = &square;
    p->draw();

    return 0;
}
```

Polymorphic function calls

Microsoft Visual Studio Debu

Drawing a circle shape.
Drawing a square shape.

A base class pointer can point to derived class objects, **but** a derived class pointer can't point to a base class object!

If we hypothetically allowed derived class pointer point to a base class object, accessing derived class-specific utilities through the derived class pointer would not be possible, as the base class object would lack the additional members and methods defined in the derived class

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

5

Example-3: Virtual Functions and Polymorphism – *cont.*

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw() { // Virtual function
        cout << "Drawing a general shape." << endl;
    }
};
class Circle : public Shape {
public:
    // 'override' is not strictly necessary
    // For clarifying your intention to human readers
    // Compiler performs additional checks
    void draw() override {
        cout << "Drawing a circle shape." << endl;
    }
};
class Square : public Shape {
public:
    void draw() override {
        cout << "Drawing a square shape." << endl;
    }
};
```

```
// Objects are received by their references
void drawShape(Shape& shape) {
    // Function call depends on the type of object
    shape.draw();
}
int main() {
    Circle circle;
    drawShape(circle);

    Square square;
    drawShape(square);

    return 0;
}
```

Microsoft Visual Studio Debu

Drawing a circle shape.
Drawing a square shape.

C++11 standard adds **override** keyword.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

6

Example-4: Object Slicing

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw() { // Virtual function
        cout << "Drawing a general shape." << endl;
    }
};
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a circle shape." << endl;
    }
};
class Square : public Shape {
public:
    void draw() override {
        cout << "Drawing a square shape." << endl;
    }
};

// Function uses call-by-value method
// Thus, it is sliced to base kind object
void drawShape(Shape shape) {
    shape.draw();
}

int main() {
    Circle circle;
    drawShape(circle);

    Square square;
    drawShape(square);

    return 0;
}
```

Microsoft Visual Studio Debug Console

```
Drawing a general shape.
Drawing a general shape.
```

Object slicing occurs when an object of a derived class is assigned to an object of a base class, resulting in the loss of the derived class-specific attributes and methods.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

7

Example-5: Virtual Functions and Polymorphism – *cont.*

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw() { // Virtual function
        cout << "Drawing a general shape." << endl;
    }
};
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a circle shape." << endl;
    }
};
class Square : public Shape {
public:
    void draw() override {
        cout << "Drawing a square shape." << endl;
    }
};

int main() {
    Shape *p;

    p = new Circle; // Runtime object is pointed to by p
    // Compiler decides which function to call
    // by looking at the type of object p points to
    p->draw();

    delete p; // Destroy runtime object

    p = new Square; // Runtime object pointed to by p
    // Compiler decides which function to call
    // by looking at the type of object p points to
    p->draw();

    delete p; // Destroy runtime object

    return 0;
}
```

Microsoft Visual Studio Debug Console

```
Drawing a circle shape.
Drawing a square shape.
```

This example presents a typical usage of runtime polymorphism.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

8

Example-6: Virtual Functions and Polymorphism – *cont.*

```
#include <iostream>
using namespace std;
class base {
public:
    virtual void vfunc() { // Virtual function
        cout << "Base class' vfunc() called." << endl;
    }
};
class derived1: public base {
public:
    // vfunc() is not overridden
};
class derived2: public derived1 {
public:
    void vfunc() override {
        cout << "Derived2 class' vfunc() called." << endl;
    }
};
class derived3: public derived2 {
public:
    // vfunc() is not overridden
};
```

```
int main() {
    base b;
    b.vfunc();

    base *p;
    p = &b;
    p->vfunc();

    derived1 d1;
    p = &d1;
    p->vfunc();

    derived2 d2;
    p = &d2;
    p->vfunc();

    derived3 d3;
    p = &d3;
    p->vfunc();

    return 0;
}
```

Microsoft Visual Studio Debug Console

```
Base class' vfunc() called.
Base class' vfunc() called.
Base class' vfunc() called.
Derived2 class' vfunc() called.
Derived2 class' vfunc() called.
```

A derived class may or may not override a virtual function. If the virtual function is not overridden in the derived class, the base class's virtual function will be called.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

9

Example-7: Pure Virtual Functions and Abstract Classes

```
// An abstract class
// contains at least one pure virtual function
class Number {
protected:
    int value;
public:
    void set_value(int v) {
        value = v;
    }
    virtual void show() = 0; // Pure virtual function
};

class Hexadecimal: public Number {
public:
    void show() override {
        cout << hex << value << endl;
    }
};

class Decimal: public Number {
public:
    void show() override {
        cout << value << endl;
    }
};

class Octal: public Number {
public:
    void show() override {
        cout << oct << value << endl;
    }
};
```

```
int main() {
    // Number n; // Can't instantiate an object
    // of abstract class type

    Number *p; // Base class pointer

    p = new Decimal;
    p->set_value(63);
    p->show();
    delete p;

    p = new Hexadecimal;
    p->set_value(63);
    p->show();
    delete p;

    p = new Octal;
    p->set_value(63);
    p->show();
    delete p;

    return 0;
}
```

show() not callable through n

Micros

```
63
3f
77
```

Pure virtual functions are not defined at base class but they **must** be defined by the deriving classes.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

10

Example-8: Static Binding at Compile Time

```
#include<iostream>
using namespace std;
int minimum(int a, int b) {
    return a > b ? b : a;
}
int minimum(char a, char b) {
    return a > b ? b : a;
}
int main() {
    int i1 = 11, i2 = 7;

    // Compiler decides the right function
    // to call by looking at its signature
    cout << "Minimum of " << i1 << " and " << i2
        << " is " << minimum(i1, i2) << "." << endl;

    char c1 = 'D', c2 = 'G';

    // Compiler decides the right function
    // to call by looking at its signature
    cout << "Minimum of " << c1 << " and " << c2
        << " is " << minimum(c1, c2) << "." << endl;

    return 0;
}
```

Function overloading

Microsoft Visual Studio Debug
Minimum of 11 and 7 is 7.
Minimum of D and G is 68.

```
#include<iostream>
using namespace std;
class Integer {
    friend ostream& operator<<(ostream& out, const Integer& r) {
        out << r.v;
        return out;
    }
    int v;
public:
    Integer() : v() {}
    Integer(int v) : v(v) {}
    void set(int v) { this->v = v; }
    int get() const { return v; }
    Integer operator+(const Integer& r) {
        return Integer(this->v + r.v);
    }
    Integer operator+(const int v) {
        return Integer(this->v + v);
    }
};
int main() {
    Integer i1, i2(3);
    i1.set(2);

    // Compiler decides the right operator function
    // to call by looking at the calling object
    cout << (i1 + i2) << endl;

    // Compiler decides the right operator function
    // to call by looking at the calling object
    cout << (i1 + 1) << endl;

    return 0;
}
```

Operator overloading

5
3

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

11

Example-9: Dynamic Binding at Runtime

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw() { // Virtual function
        cout << "Drawing a general shape." << endl;
    }
};
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a circle shape." << endl;
    }
};
class Square : public Shape {
public:
    void draw() override {
        cout << "Drawing a square shape." << endl;
    }
};
```

```
int main() {
    Shape *p;

    p = new Circle; // Runtime object is pointed to by p
    // Compiler decides which function to call
    // by looking at the type of object p points to
    p->draw();

    delete p; // Destroy runtime object

    p = new Square; // Runtime object pointed to by p
    // Compiler decides which function to call
    // by looking at the type of object p points to
    p->draw();

    delete p; // Destroy runtime object

    return 0;
}
```

Microsoft Visual Studio Deb
Drawing a circle shape.
Drawing a square shape.

In C++, dynamic binding is primarily achieved through class inheritance and polymorphism.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

12

Example-10: Static/Dynamic Binding Syntax Issues

```
#include<iostream>
using namespace std;
class base {
public:
    virtual void vfunc() {
        cout << "Base vfunc() is called." << endl;
    }
};
class derived : public base {
public:
    void vfunc() override {
        cout << "Derived vfunc() is called." << endl;
    }
};
```

In C++, member access using the scope resolution operator works with static binding, meaning the compiler resolves the function call at compile time based on the type specified to the left of the scope resolution operator. In this case, bp is a pointer to the base class, and derived::vfunc() is an attempt to directly access the vfunc() function of the derived class using bp. Since bp is a pointer to the base class, and the base class doesn't have a member function named vfunc() in the derived class's scope, the compiler will generate an error.

```
int main() {
    base b;
    b.vfunc();           // Static binding

    base *bp;
    bp = new derived;
    bp->base::vfunc();    // Static binding
    // bp->derived::vfunc(); // error !
    bp->vfunc();          // Dynamic binding
    delete bp;

    derived* dp;
    // dp = new base;      // error !
    dp = new derived;
    dp->base::vfunc();     // Static binding
    dp->derived::vfunc(); // Static binding
    dp->vfunc();          // Dynamic binding
    delete dp;

    return 0;
}
```

Early binding and static binding mean the same thing and refer to compile-time resolution, while late binding and dynamic binding also mean the same thing and refer to runtime resolution.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

13

Example-11: Virtual Destructor

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base class object constructed." << endl;
    }
    // Virtual destructor
    virtual ~Base() {
        cout << "Base class object destructed." << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived class object constructed." << endl;
    }
    // Destructor
    ~Derived() {
        cout << "Derived class object destructed." << endl;
    }
};
```

```
int main() {
    // Create a derived class object
    // and let it be pointed to by base class pointer
    Base* ptr = new Derived();

    // To properly initiate calls to all destructors
    delete ptr;

    return 0;
}
```

Microsoft Visual Studio Debug Console

```
Base class object constructed.
Derived class object constructed.
Derived class object destructed.
Base class object destructed.
```

Deleting 'virtual' keyword

```
// Destructor
~Base() {
    cout << "Base Destructor" << endl;
}
```

Microsoft Visual Studio

```
Base Constructor
Derived Constructor
Base Destructor
```

Correct Resource Deallocation

When a derived class object is deleted through a base class pointer, a virtual destructor ensures that the derived class's destructor is called, followed by the base class's destructor. This ensures that all resources (like dynamically allocated memory) in both the base and derived classes are properly released.

Lecture-10/11

COMP218 by Dr. Ferhun Yorgancıoğlu

14

Example-12: Runtime Polymorphism in Action

```
#include <iostream>
#include <cmath>
#define PI 3.14
using namespace std;

// Base class: Point
class Point {
public:
    Point(int x = 0, int y = 0) : x(x), y(y) {}
    void setCoordinates(int x, int y) { ... }
    void getCoordinates(int &x, int &y) const { ... }
    // Virtual function
    virtual void display() const {
        cout << "Point(" << x << ", " << y << ")" << endl;
    }
protected:
    int x, y;
};

// Derived class: Circle
class Circle : public Point {
public:
    Circle(int x = 0, int y = 0, int radius = 1) : Point(x, y), radius(radius) {}
    void setRadius(int radius) { ... }
    int getRadius() const { ... }
    double area() const { ... }
    void display() const override {
        cout << "Circle(" << x << ", " << y << " radius: " << radius << ")" << endl;
    }
protected:
    int radius;
};

// Derived class: Cylinder
class Cylinder : public Circle {
public:
    Cylinder(int x = 0, int y = 0, int radius = 1, int height = 1)
        : Circle(x, y, radius), height(height) {}
    void setHeight(int height) { ... }
    int getHeight() const { ... }
    double volume() const { ... }
    void display() const override {
        cout << "Cylinder(" << x << ", " << y << " radius: " << radius << " height: " << height << ")" << endl;
    }
private:
    int height;
};

int main() {
    Point point(2, 2);
    Circle circle(3, 3, 2);
    Cylinder cylinder(4, 4, 3, 5);

    Point *shapes[3] = { &point, &circle, &cylinder };
    for (int i = 0; i < 3; ++i)
        shapes[i]->display(); // Runtime polymorphism
    return 0;
}
```

Microsoft Visual Studio Debug Console

```
Point(2, 2)
Circle(3, 3, radius: 2)
Cylinder(4, 4, radius: 3, height: 5)
```